Software Engineering — Project Proposal

AI-Powered Phishing URL Detector

Real-Time Threat Detection Using Machine Learning

Submitted by

Name	Roll No.
Rahul Arora	1024170368
Madhav Singla	1024170356
Rakshat Mastana	1024170345

Batch: **3Q33**

Presented to

Mr. Hardik Gujral

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala

Software Engineering — 5th Semester

August 20, 2026

Acknowledgement

We would like to express our sincere gratitude to **Mr. Hardik Gujral** for his guidance and valuable feedback throughout the conceptualization of this project. We also thank the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, for providing the resources and platform necessary to undertake this work as part of the Software Engineering course curriculum.

Abstract

Phishing attacks remain among the most prevalent and financially damaging forms of cybercrime, frequently serving as the initial entry point for larger-scale security breaches. Existing protective mechanisms largely depend on static blocklists, which are inherently reactive and ineffective against newly created, previously unseen malicious URLs. This proposal outlines the design and development of a full-stack web application — the **AI-Powered Phishing URL Detector** — that performs real-time, machine-learning-based analysis of submitted URLs to assess their likelihood of being phishing attempts. The system extracts a set of lexical and metadata-derived features from a given URL, evaluates them using a trained classification model, and returns an explainable risk assessment to the user, rather than a simple binary verdict. The application is built using a React frontend, an Express.js backend, a Python-based machine learning module, and MongoDB for persistent storage. This document details the problem motivating the project, the proposed system architecture, the technology stack, the development methodology, and the anticipated timeline and outcomes.

Contents

1 Introduction

Phishing is a form of social-engineering attack in which an adversary impersonates a legitimate entity — typically through a deceptive website or communication — in order to trick a victim into disclosing sensitive information such as login credentials, financial details, or personal data. Unlike attacks that exploit software vulnerabilities directly, phishing exploits human trust and inattention, making it difficult to fully prevent through technical means alone.

The scale of the problem is substantial. Industry estimates place global losses attributable to phishing at over $25 billion annually, and phishing is consistently cited as the initial attack vector in the majority of reported data breaches. With the growing accessibility of generative AI tools, attackers are now able to produce convincing phishing websites, emails, and even voice content at a pace and scale that traditional defenses struggle to match.

The most common consumer-facing defense against phishing — the static blocklist, used by most browsers and antivirus tools — depends on a URL having already been identified, reported, and cataloged as malicious before it can be flagged. This creates an inherent window of vulnerability during which newly created phishing sites remain completely undetected. Addressing this gap requires detection mechanisms capable of assessing a URL's risk based on its own inherent characteristics, at the moment it is encountered, rather than relying solely on historical records.

This project proposes such a system: a web-based application that uses a trained machine learning model to assess the phishing risk of any submitted URL in real time, and — critically — explains the specific factors contributing to that assessment, rather than returning an opaque classification.

2 Problem Statement

Existing phishing-detection tools made available to end users suffer from several notable limitations:

(i) **Reactivity:** Blocklist-based systems can only detect URLs that have already been reported and cataloged, offering no protection against newly registered (—zero-hour‖) phishing domains.

(ii) **Lack of transparency:** Most existing tools return a binary safe/unsafe verdict without explaining which characteristics of the URL informed that decision, limiting user trust and understanding.

(iii) **Limited accessibility:** Robust phishing detection capabilities are often embedded within enterprise security suites or browser-specific extensions, rather than being available as an independent, easily usable tool.

(iv) **Scalability against AI-generated threats:** As phishing content generation becomes increasingly automated, static, manually curated blocklists are unlikely to scale at a comparable pace.

There is a clear need for a lightweight, explainable, and independently accessible tool capable of assessing URL risk in real time using measurable, extractable features of the URL itself.

3 Objectives

The primary objectives of this project are as follows:

1. To design and train a machine learning classification model capable of distinguishing phishing URLs from legitimate ones, based on structural and metadata-derived features.
2. To develop a RESTful backend service that performs real-time feature extraction and model inference for any user-submitted URL.
3. To build a responsive, user-friendly web frontend that allows users to submit URLs, view detailed risk assessments, and review their historical scan activity.
4. To implement secure, token-based user authentication, enabling scan history to be maintained privately on a per-user basis.
5. To present model predictions in an explainable format, clearly identifying which specific features contributed to a given risk score.
6. To evaluate and compare multiple candidate machine learning models, selecting the one offering the best balance of accuracy and interpretability.

4 Scope of the Project

The scope of this project is limited to the detection of phishing risk based on **URL-level features** — that is, characteristics derivable from the URL string itself and its associated domain metadata (e.g., domain age, certificate status). The system does not perform deep content analysis of a website's rendered page (e.g., visual similarity detection, embedded script analysis), nor does it attempt to block or intercept network traffic at the browser or operating-system level. The deliverable is a standalone web application; browser-extension integration is identified as a possible future extension but is not part of the core deliverable for this course project.

5 Literature Review and Existing Systems

Phishing detection has been studied extensively in both industry and academic contexts, and existing approaches can be broadly grouped into the following categories:

5.1 Blocklist-Based Detection

Services such as Google Safe Browsing maintain continuously updated databases of URLs confirmed to host malicious or deceptive content. Browsers and security software query these databases to warn users before they access a flagged site. While highly reliable for known threats, this approach is inherently reactive, offering zero protection against a phishing site in the window between its creation and its eventual reporting.

5.2 Heuristic and Rule-Based Detection

Some tools apply manually defined rules — for example, flagging URLs that use an IP address in place of a domain name, or that contain an excessive number of subdomains. While these rules can catch certain classes of phishing attempts, they are relatively easy for attackers to circumvent once the rules become known, and they do not adapt automatically to new attack patterns.

5.3 Machine-Learning-Based Detection

A substantial body of academic research applies supervised machine learning models — including Logistic Regression, Random Forest, Support Vector Machines, and Gradient Boosting classifiers — to lexical and metadata features of URLs, frequently reporting classification accuracies exceeding 90% on benchmark phishing datasets. This approach generalizes better to previously unseen URLs than purely rule-based systems, since the model learns patterns from data rather than relying on a fixed, hand-crafted rule set.

5.4 Gap Identified

While the machine-learning-based approach is well established in academic literature, it is comparatively rare to find it implemented as part of a complete, usable, full-stack web application accessible to a general end user — most academic implementations remain confined to offline model evaluation in a notebook environment. Furthermore, few freely accessible tools provide **explainable** output, instead returning only a bare classification. This project directly addresses both gaps: integrating a trained classifier into a live, real-time web application, and surfacing the specific contributing features behind each prediction.

6 Requirement Analysis

6.1 Functional Requirements

- The system shall allow a user to register for a new account and log in using a valid email and password.
- The system shall allow an authenticated user to submit a URL for phishing risk analysis.
- The system shall extract a defined set of features from the submitted URL and return a numeric risk score (0–100), a categorical verdict (Safe, Suspicious, or Phishing), and a list of contributing risk factors.
- The system shall persist each scan result, associated with the submitting user, for later retrieval.
- The system shall allow a user to view a history of their previous scans, and to view full detail (including risk factors) for any individual past scan.
- The system shall allow a user to delete an individual scan record from their history.

- The system shall display aggregate statistics (e.g., total scans performed, breakdown by verdict) on a summary dashboard.

6.2 Non-Functional Requirements

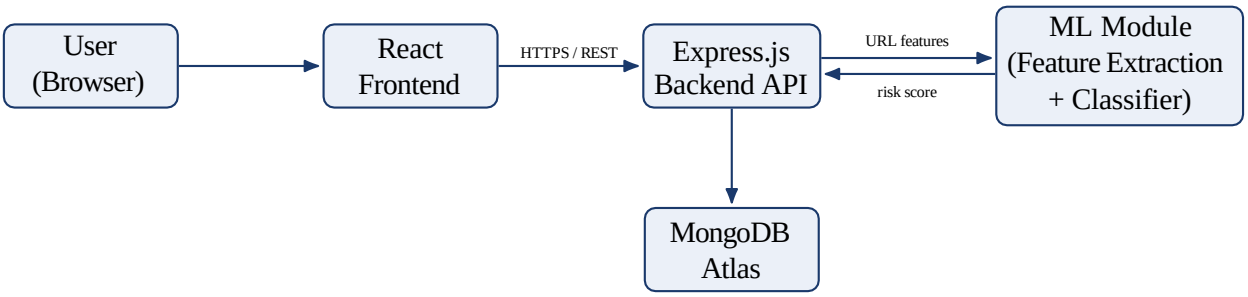
- **Performance:** A submitted URL should return a risk assessment within a few seconds under normal conditions, to preserve the real-time user experience.
- **Security:** User passwords must be stored using a secure hashing mechanism; authenticated routes must validate a token on every request; database credentials must never be committed to version control.
- **Usability:** The interface should present risk information in a clear, non-technical manner, understandable to users without a security background.
- **Availability:** The application should be deployable such that it remains accessible for demonstration and evaluation without requiring manual server management during use.
- **Maintainability:** The codebase should be modular, with a clearly separated frontend, backend, and machine learning component, each independently testable.

7 Proposed System

7.1 System Overview

The proposed system follows a three-tier architecture consisting of a presentation layer (React frontend), an application layer (Express.js REST API), and a data/intelligence layer comprising both the MongoDB database and the Python-based machine learning inference component. This separation allows each layer to be developed, tested, and iterated upon independently, provided all layers adhere to a shared, predefined API contract.

7.2 System Architecture



7.3 Data Flow

1. The user submits a URL through the React frontend.

2. The frontend sends an authenticated `POST` request to the backend's `/api/scan` endpoint.
3. The backend extracts a defined feature set from the URL (e.g., domain age, HTTPS status, lexical patterns) and passes them to the trained classification model.
4. The model returns a probability score, which the backend converts into a risk score and categorical verdict.
5. The backend persists the result to MongoDB, associated with the requesting user, and returns the full result to the frontend.
6. The frontend renders the risk score, verdict, and contributing factors to the user.

7.4 Key Features

- **Real-time URL scanning** — every submission is analyzed live; results are never served from a static, precomputed list.
- **Explainable risk output** — each result includes the specific features (e.g., absence of HTTPS, recently registered domain, similarity to a known brand domain) that contributed to the score.
- **Secure authentication** — token-based login ensures scan history remains private to each user.
- **Scan history dashboard** — a chronological, reviewable record of all past scans, with per-record deletion.
- **Summary statistics** — an at-a-glance breakdown of a user's scanning activity by verdict category.

7.5 Machine Learning Approach

The classification model will be trained on a labeled, publicly available dataset containing both phishing and legitimate URLs. The following categories of features are proposed for extraction:

- URL length and subdomain count
- Presence of an IP address in place of a domain name
- Presence and validity of HTTPS
- Domain age, retrieved via WHOIS lookup
- Lexical similarity to well-known brand domains (typosquatting detection)
- Presence of suspicious keywords within the URL path
- Number of special characters (e.g., `@`, `-`) in the URL

Three candidate models are proposed for evaluation: **Logistic Regression** (as a fast, interpretable baseline), **Random Forest** (expected to offer strong accuracy on mixed categorical/numerical features), and a **small feed-forward neural network** (evaluated for comparison

against the classical approaches). Each model will be evaluated using accuracy, precision, recall, and F1-score on a held-out test set, and the best-performing model — balancing predictive performance against interpretability — will be selected for deployment.

8 Technology Stack

Layer	Technology
Frontend	React (with Context API for state management), REST-based API communication
Backend	Node.js with Express.js
Machine Learning	Python, scikit-learn, pandas
Database	MongoDB Atlas (accessed via Mongoose ODM)
Version Control & Collaboration	Git and GitHub, with a branch-per-feature and pull-request workflow
Development Tools	Visual Studio Code, Postman / Thunder Client (API testing)

9 Methodology / Development Approach

The project will follow an iterative, modular development approach, structured around four broad phases:

1. **Data collection and preprocessing.** A labeled phishing/legitimate URL dataset will be sourced, cleaned, and used to engineer the feature set described in Section 8.5.
2. **Model development.** Candidate models will be trained, tuned, and evaluated, with the best-performing model selected and packaged for integration into the backend.
3. **Parallel frontend and backend development.** A shared API contract, defining exact request and response formats for every endpoint, will be finalized before implementation begins. This allows backend development (API routes, authentication, ML integration) and frontend development (UI, forms, dashboards, built initially against mock data matching the contract) to proceed concurrently rather than sequentially.
4. **Integration and testing.** Once both halves are functional independently, the frontend is connected to the live backend, followed by end-to-end functional testing across the complete user flow — registration, login, URL scanning, and history management.

This approach was deliberately chosen to allow team members with differing areas of strength to contribute meaningfully and concurrently, rather than being blocked on sequential handoffs between layers.

10 Team Composition and Role Distribution

Name	Roll No.	Primary Responsibility
Rahul Arora	1024170368	Backend development, machine learning model design and training, system integration
Rakshat Mastana	1024170345	Frontend development (React UI, dashboards, scan and history views)
Madhav Singla	1024170356	Documentation, functional testing, and dashboard/summary features

All source code will be maintained in a shared, version-controlled GitHub repository, organized into clearly separated `frontend`, `backend`, `ml`, and `docs` directories, with development conducted via feature branches and reviewed through pull requests prior to merging into the main branch.

11 Proposed Timeline

Week(s)	Planned Activity
Week 1	Dataset collection, feature engineering, API contract finalization, repository setup
Weeks 2–3	Model training and evaluation; backend API and authentication development
Weeks 3–4	Frontend development (conducted in parallel, against the finalized API contract)
Week 5	Integration of frontend and backend; resolution of integration issues
Week 6	Functional testing, bug fixes, and refinement
Week 7	Final documentation, report preparation, and presentation rehearsal

12 Testing Plan

The system will be tested at multiple levels:

- **Unit testing:** individual backend routes will be tested in isolation using Postman/Thunder Client, verifying correct responses for both valid and invalid inputs.
- **Model evaluation:** the trained classifier will be evaluated on a held-out test split using standard classification metrics (accuracy, precision, recall, F1-score) prior to integration.
- **Integration testing:** once connected, the complete flow — from URL submission through to displayed result and stored history — will be manually verified.

- **User acceptance testing:** team members will use the completed application as end users would, submitting a mix of known-safe and known-phishing URLs to sanity-check real-world behavior.

13 Risk Analysis

Risk	Likelihood	Mitigation
Model accuracy lower than expected on real-world URLs	Medium	Evaluate multiple models; expand/refine feature set if initial accuracy is insufficient
Integration mismatches between frontend and backend	Medium	Finalize and strictly follow a shared API contract before implementation begins
Uneven team contribution due to differing skill levels	Medium	Clearly scoped role distribution, with regular check-ins and shared documentation
External API dependency (e.g., WHOIS lookups) rate-limited or unavailable	Low–Medium	Cache lookups where possible; degrade gracefully by excluding that feature if unavailable

14 Expected Outcomes

Upon completion, this project will deliver a fully functional, deployed web application in which a user can register, securely log in, submit any URL, and receive an instant, explainable phishing-risk assessment, along with a persistent, reviewable history of past scans. Beyond the functional deliverable, the project is expected to demonstrate practical, integrated application of machine learning within a complete full-stack system — spanning frontend development, backend API design, authentication, database persistence, and applied data science — addressing a problem of genuine and growing real-world relevance.

15 Future Scope

While outside the core scope of this course project, several extensions are identified as natural next steps for the system: (i) a browser extension performing real-time scanning as a user navigates the web, rather than requiring manual URL submission; (ii) expansion of the feature set to include lightweight analysis of a page's rendered content, in addition to URL-level features; (iii) a shared organizational dashboard allowing aggregated visibility into scanning activity across multiple users; and (iv) periodic retraining of the model as new phishing patterns emerge, to maintain detection accuracy over time.

16 Conclusion

Phishing continues to be the most common initial vector for cyberattacks, and the scale of the threat is expected to grow further as generative AI lowers the barrier to producing convincing malicious content. This project proposes the design and development of a real-time, explainable, machine-learning-based phishing detection system, built on an accessible and well-established full-stack architecture. Beyond its practical utility, the project offers a comprehensive demonstration of the integration of applied machine learning with modern full-stack software engineering practices, fulfilling the objectives of the Software Engineering course while addressing a problem of genuine contemporary significance.

References

1. Anti-Phishing Working Group (APWG), *Phishing Activity Trends Reports*, https://apwg.org
2. Cybersecurity and Infrastructure Security Agency (CISA), *Phishing Guidance*, https://www.cisa.gov
3. IBM Security, *Cost of a Data Breach Report*, IBM Corporation.
4. Google Safe Browsing, *Transparency Report*, https://safebrowsing.google.com
5. scikit-learn documentation, https://scikit-learn.org
6. MongoDB Documentation, https://www.mongodb.com/docs
7. Express.js Documentation, https://expressjs.com
8. React Documentation, https://react.dev